

Fooling Automated Surveillance Cameras

Adversarial patches to attack person detection

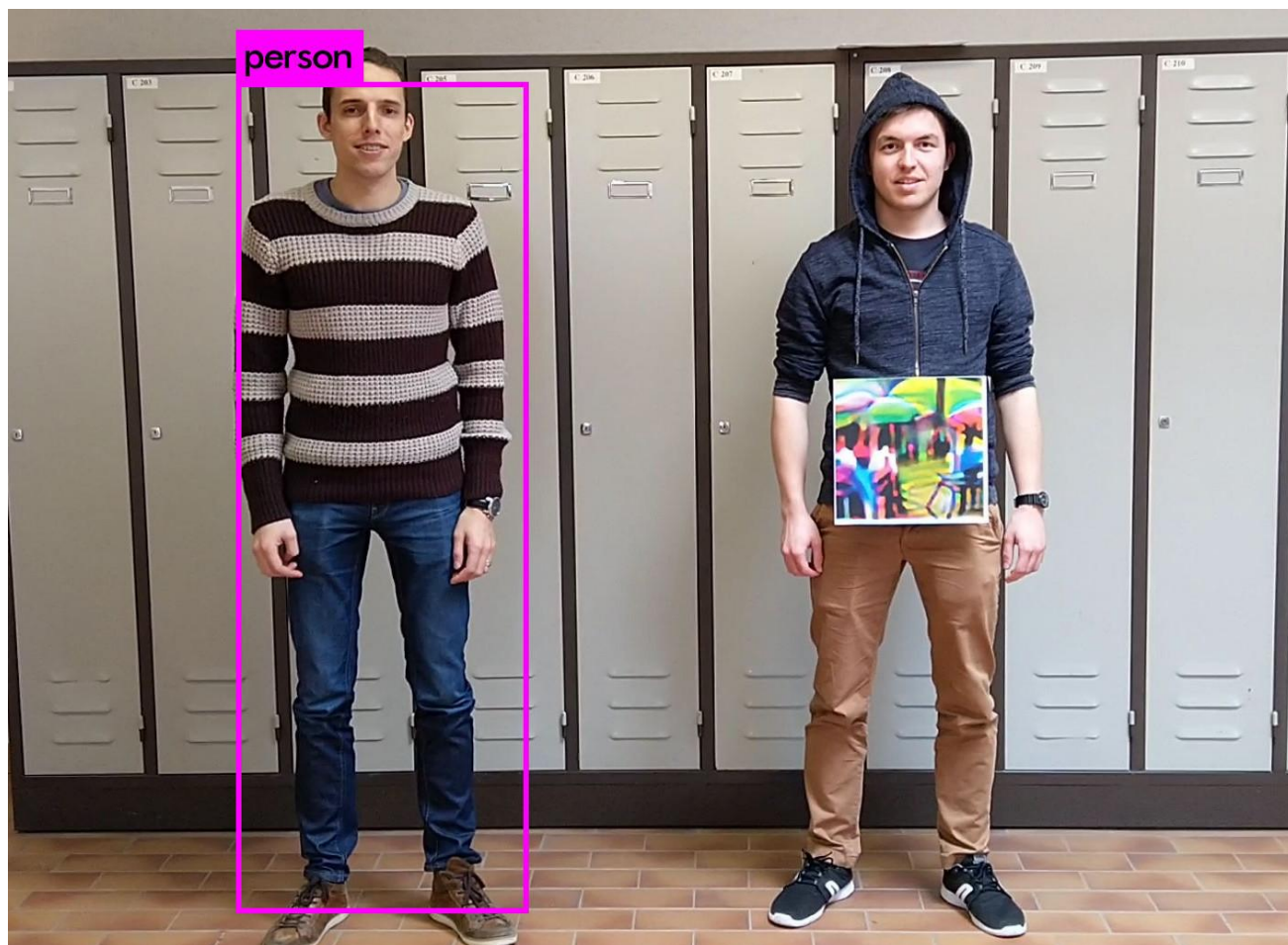
Simen Thys, Wiebe Van Ranst, Toon Goedemé

KU Leuven · CVPR Workshops (CV-COPS) 2019



person: not found

Two people. The detector sees only one.



Left

No patch → YOLOv2 draws a "person" box.

Right

Holds a printed $\sim 40 \times 40$ cm patch → no box at all.

**The camera still records him.
So what failed?**

Camera → **Detector** → Alarm

The attack targets this step

Let's start from the ground up

1

How YOLO finds people

What goes in, what comes out, and how it learns

2

How to make a patch that hides you

Freeze the detector, change only the patch

3

Results

On test images, then in front of a real camera

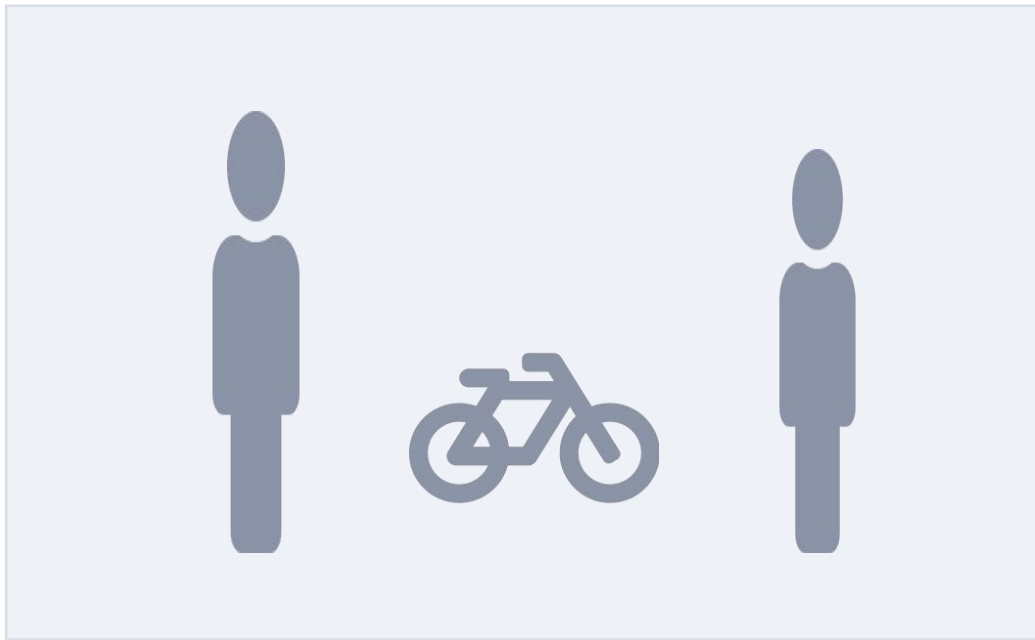
1

How YOLO finds people

To fool a detector, first see what it outputs.

Classification vs. detection: "what" vs. "what and where"

Image classification

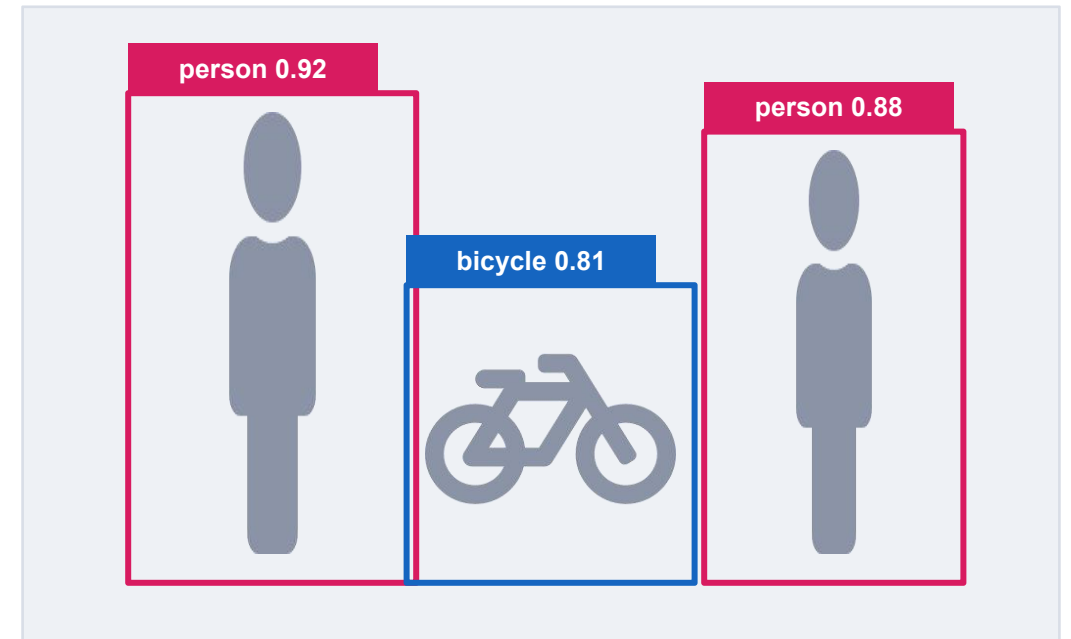


person

One label for the whole image.

For a camera, the person box is what triggers the alarm.

Object detection



A box + a label + a score for every object.

 [Live demo: object detection on your webcam](#)

What YOLO learns from: an image plus a list of boxes



Input: the image

Label: one line per object

```
class      : person
x, y       : 0.58, 0.51
w, h       : 0.45, 0.90
```

class — what the object is

x, y — center of the box

w, h — width and height

Training

Input

the image

Target

the boxes a human drew

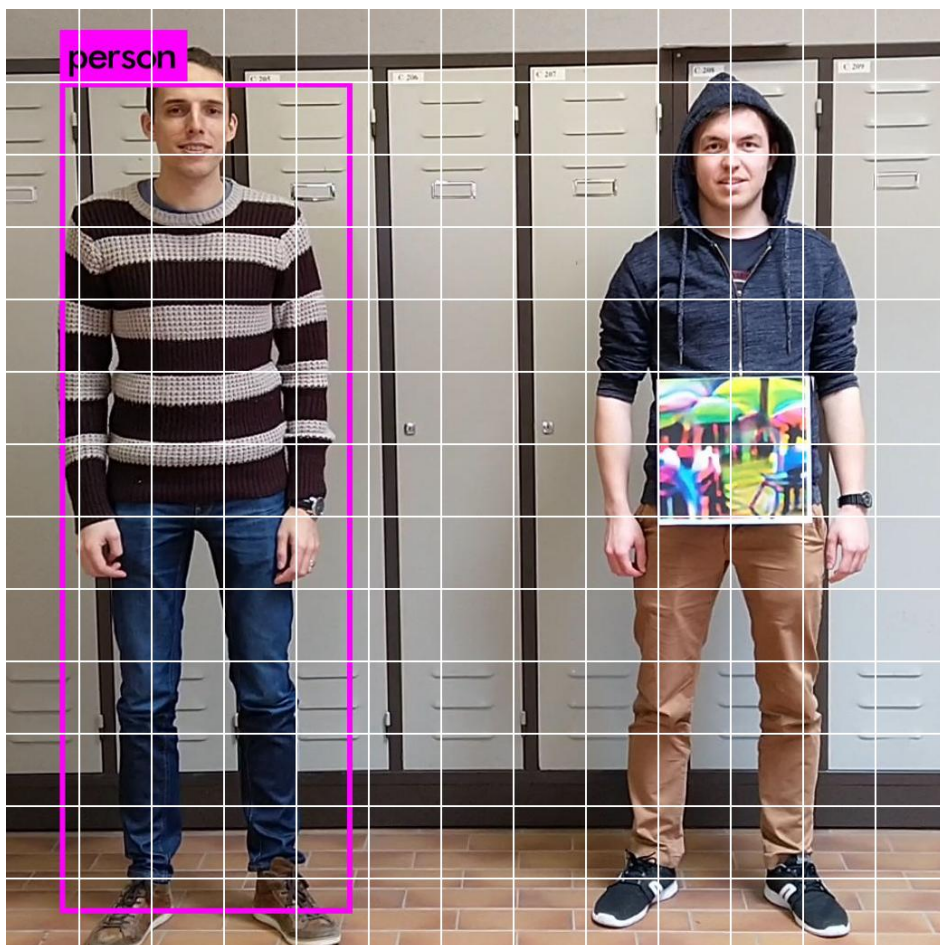
Dataset

MS COCO: 80 classes, "person" is one of them



<https://docs.ultralytics.com/datasets/detect/coco>

What YOLOv2 outputs: 845 candidate boxes, every time



13 × 13 grid over the image (illustration)

- The image is divided into a 13 × 13 grid.
 - Each cell makes 5 guesses ("anchors").
- 13 × 13 × 5 = 845** candidate boxes per image.

Each candidate is 85 numbers:

x	y	w	h	p_obj	p_person	p_bicycle	...	p_toothbrush
---	---	---	---	-------	----------	-----------	-----	--------------

Where is the box?

4 numbers

Is anything here?

1 number: objectness

Which class?

80 numbers, one per class

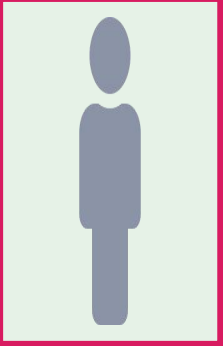
Remember p_obj and p_person. The whole attack is about these two numbers.

A box is drawn only if its score passes a threshold

$$\text{score}_{\text{person}} = p_{\text{obj}} \times p_{\text{person}}$$

Keep the box only if the score is above a threshold (the paper uses 0.4).

person 0.81



$$0.9 \times 0.9 = 0.81$$

$p_{\text{obj}} \times p_{\text{person}}$

above 0.4 → box drawn



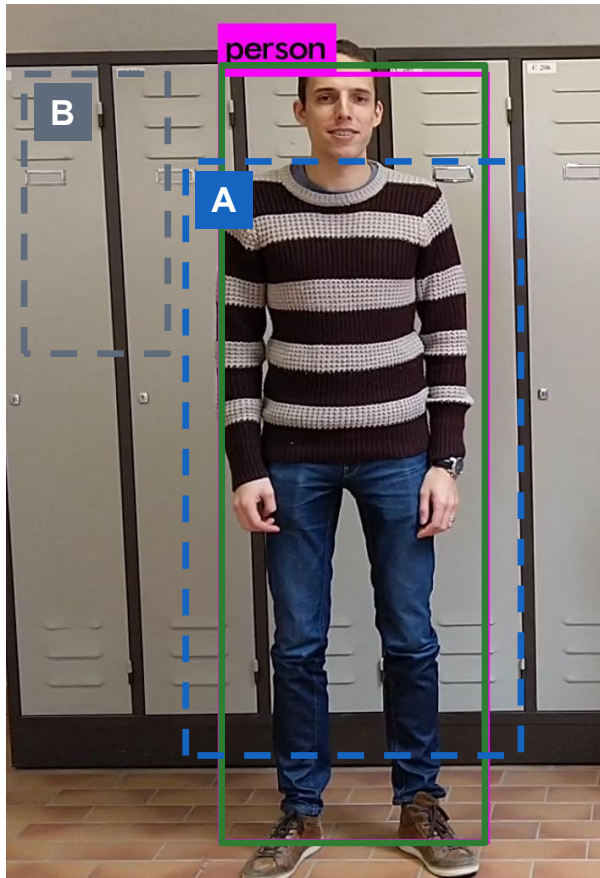
$$0.1 \times 0.9 = 0.09$$

$p_{\text{obj}} \times p_{\text{person}}$

below 0.4 → nothing reported

To hide a person, push the score below the threshold. It's enough to push down one of the two factors.

The loss on one photo



— label — guess A — guess B

Guess	Model says	Should be	Loss term
A	box too short, too wide	the green box	Where?
A	$p_{\text{obj}} = 0.6$	≈ 1	Something here?
A	$p_{\text{person}} = 0.7$	1	What is it?
B	$p_{\text{obj}} = 0.3$ (lockers)	0	Nothing here?

Gradient descent changes the weights until every term is small.



[Interactive: watch a network's loss go down \(TensorFlow Playground\)](#)

Same machinery, different knob: training vs. attacking

Training YOLO

Fixed	the images and labels
Changed	the network weights
Goal	make the loss small: the detector is right

This attack

Fixed	the network weights (frozen)
Changed	only the patch pixels
Goal	make person scores small: the detector misses people

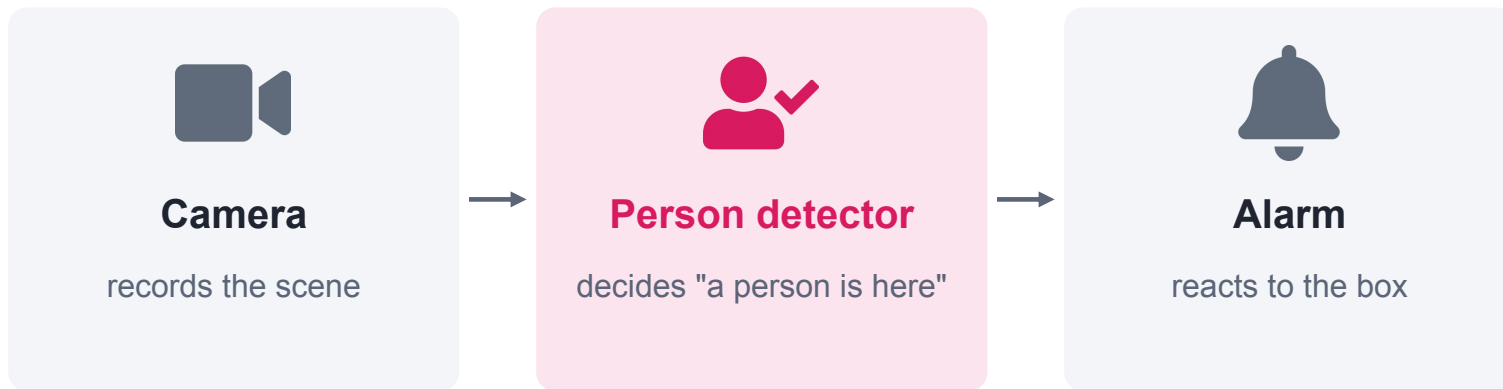
Both use gradients and backpropagation.

2

How to make a patch that hides you

Keep the detector fixed. Change only what it sees.

Why attack a person detector?



Detectors are tested on normal images. Nobody in the test set is trying to fool them. A burglar is.

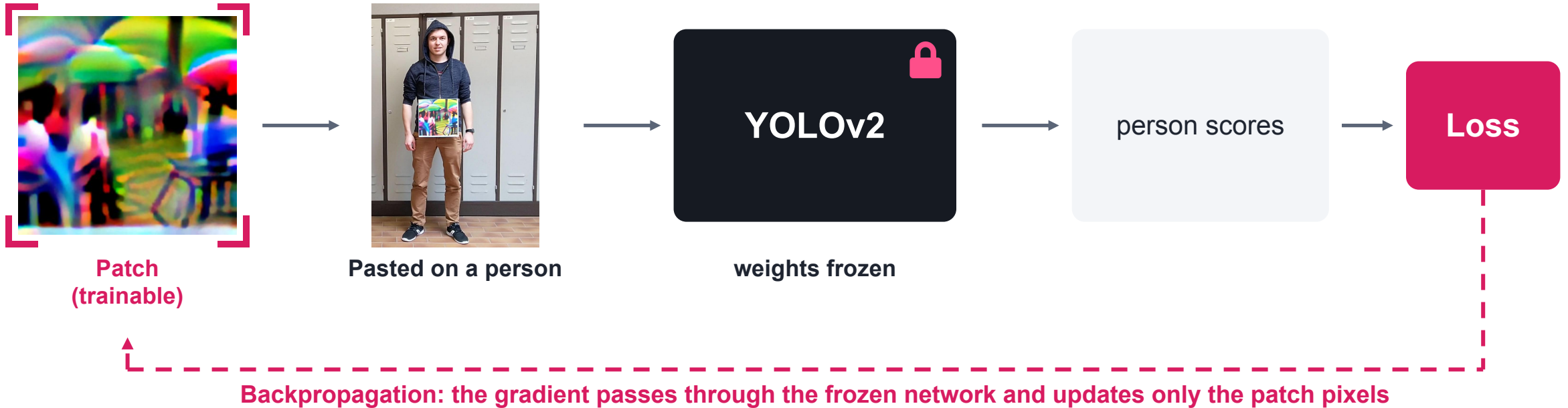
Earlier physical attacks fooled models on stop signs, which always look the same. People don't: clothes, pose, background all vary.

Threat model

An intruder holds a small printed board toward the camera and walks past without triggering the alarm.



Freeze the detector. Optimize only the patch.

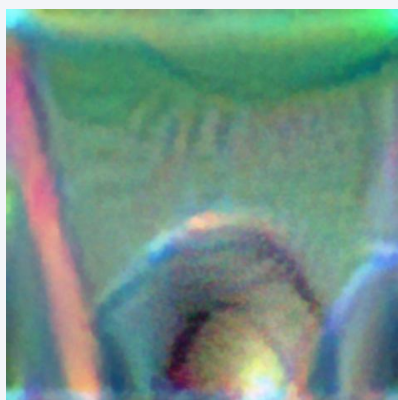


- The patch starts as random pixels.
- Needs the model's gradients → a white-box attack.
- Once trained, you just print it.

Three targets from one score: $p_{obj} \times p_{person}$

CLS

$$L_{cls} = \max_i p_{person,i}$$



"It's not a person."

Paper Fig. 4d

OBJ

$$L_{obj} = \max_i p_{obj,i}$$



"There's nothing here."

Paper Fig. 4c

OBJ-CLS

$$L_{obj \cdot cls} = \max_i (p_{obj,i} \times p_{person,i})$$



"Not a person, and nothing here."

Paper Fig. 4a

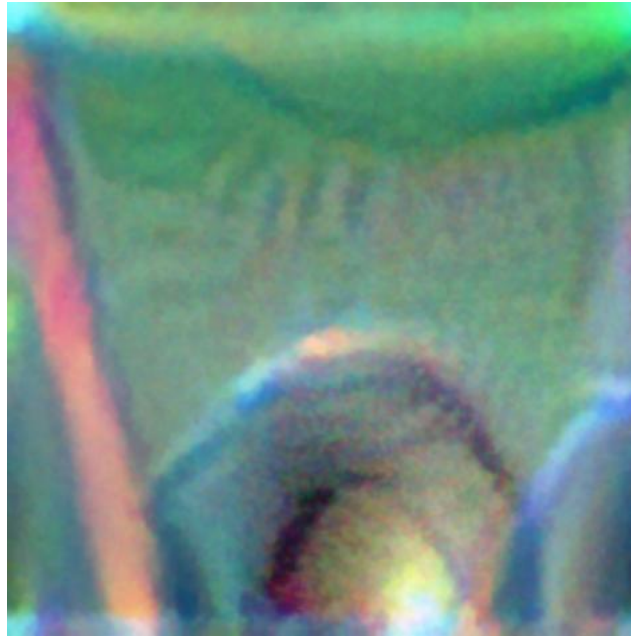
Why max? Push down the most confident of the 845 guesses. If even that one is below the threshold, no box survives.

The class-based patches turn into other objects



OBJ-CLS

Looks like a teddy bear, and YOLO sees one.



CLS

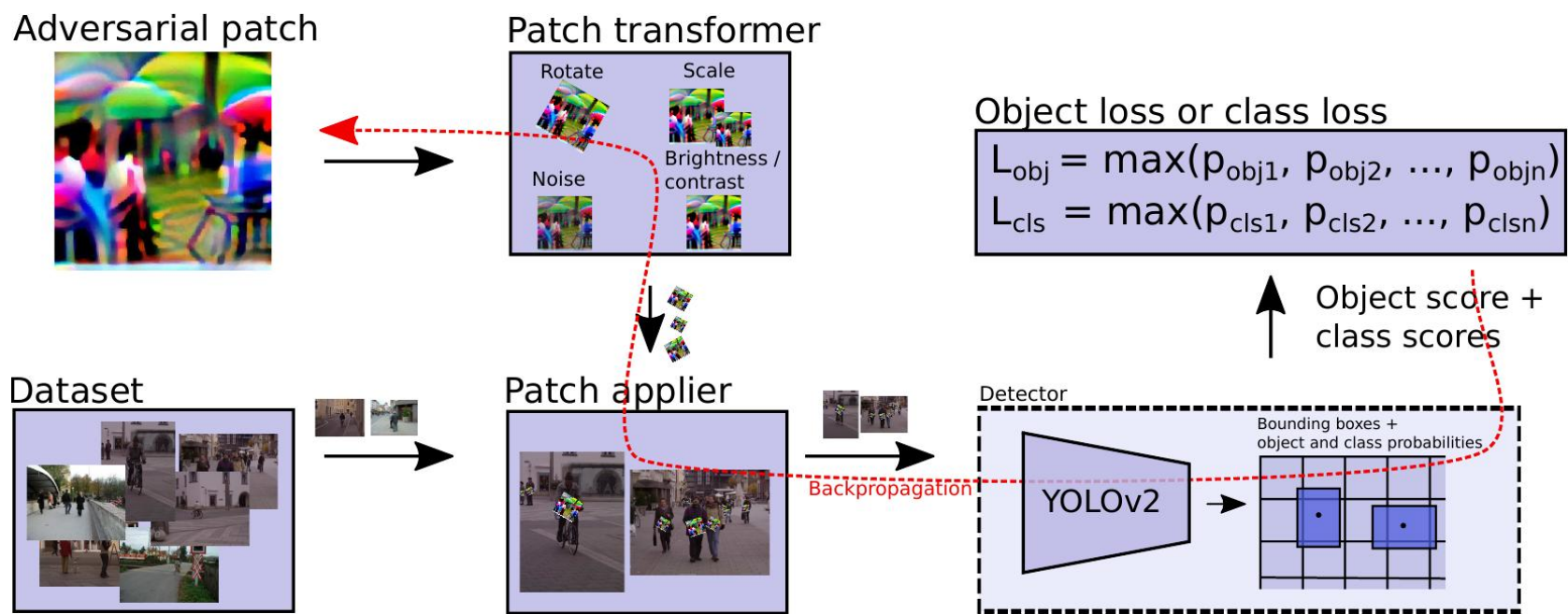
Also pushes "person" toward another class.



OBJ

Not any object. It just says "nothing here".

The whole pipeline in one picture



- 1** Randomly transform the patch
- 2** Paste it on each person YOLOv2 finds
- 3** Run the batch through frozen YOLOv2
- 4** Compute the loss
- 5** Backprop → update only the patch

Repeat over many batches of pedestrian photos (Inria dataset).

Every training step sees a slightly different patch



original

rotate -18° rotate $+15^\circ$ scale $0.7\times$

Why?

A real camera never sees the exact same patch: the angle, distance, lighting and noise change.

What they do

- Rotate up to $\pm 20^\circ$
- Scale up and down
- Add noise
- Change brightness and contrast



+ noise



darker



brighter



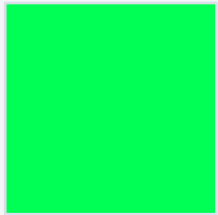
all at once

So the patch must work on average over all these versions.

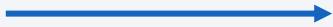
Two extra terms so the patch still works on paper

Printable colors (NPS)

Why: a printer can't reproduce every screen color.



screen color



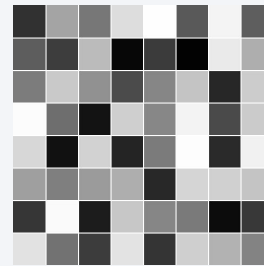
nearest printable

$$L_{nps} = \sum_{p \in P} \min_{c \in C} |p - c|$$

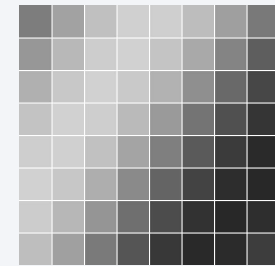
Penalize each pixel's distance to the nearest printable color.

Smooth patterns (TV)

Why: printing and cameras blur fine pixel noise.



noisy: high TV



smooth: low TV

$$L_{tv} = \sum_{i,j} \sqrt{(p_{i,j} - p_{i+1,j})^2 + (p_{i,j} - p_{i,j+1})^2}$$

Penalize differences between neighboring pixels.

The total loss: hide + printable + smooth

$$L = L_{obj} + \alpha L_{nps} + \beta L_{tv}$$

L_{obj}

Hide

highest objectness on the patched images

$\alpha \cdot L_{nps}$

Printable

distance to printable colors

$\beta \cdot L_{tv}$

Smooth

differences between neighboring pixels

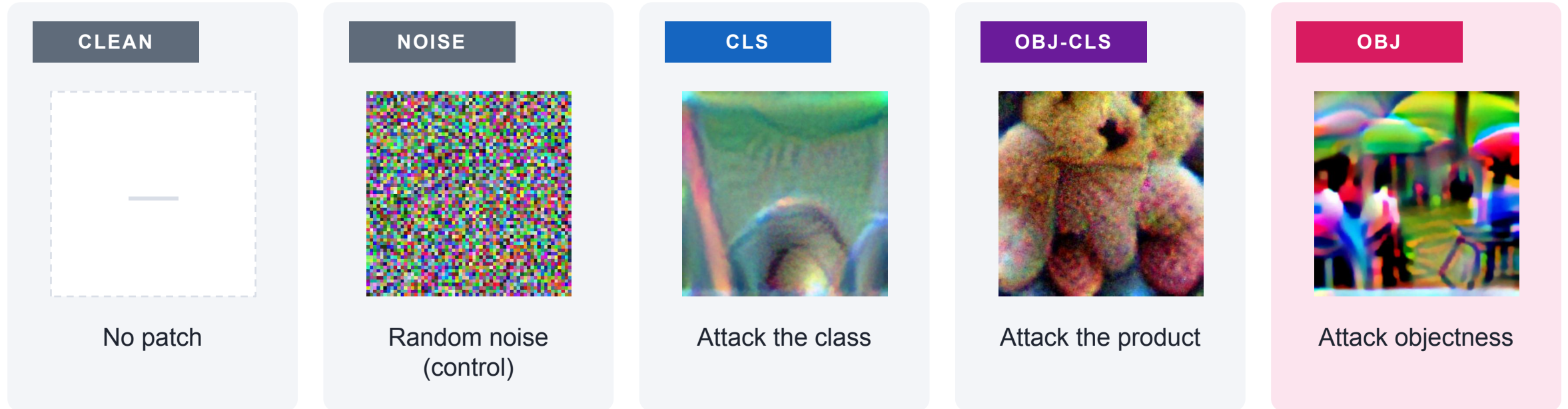
α and β balance the three goals. Only the patch is optimized (Adam); YOLOv2 stays frozen.

3

Results

First on test images, then in front of a real camera.

Five conditions on the Inria test images

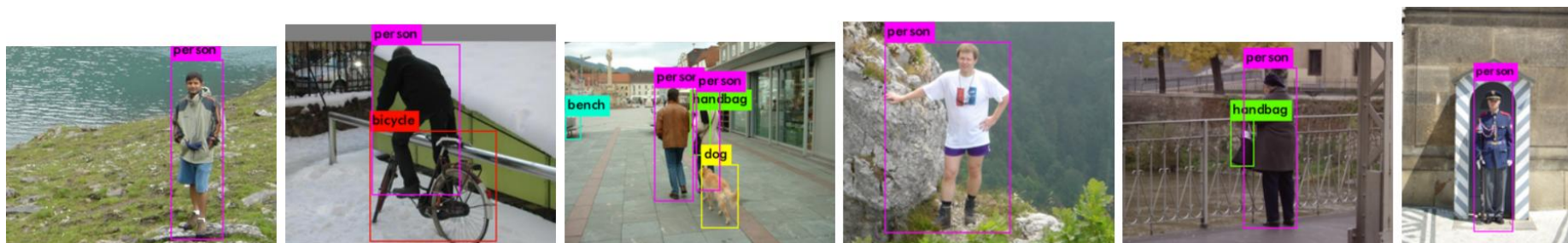


The patch is pasted on each person in the test images.

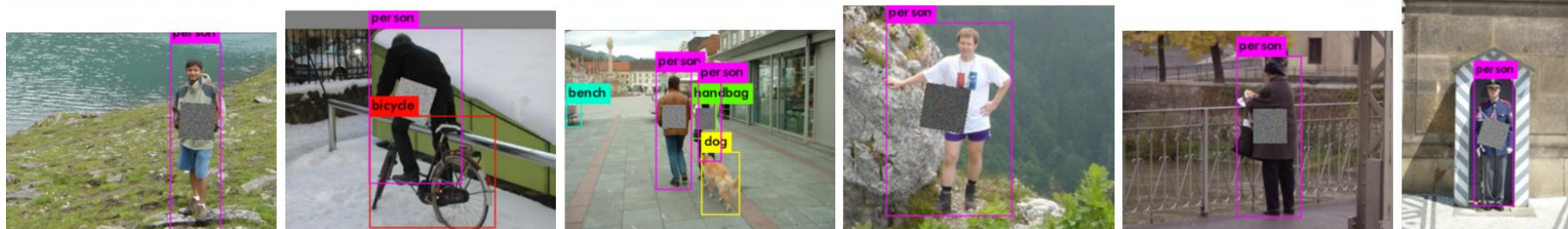
Reference = the people YOLOv2 finds with no patch, so CLEAN is 100% by definition.

What it looks like on the test images

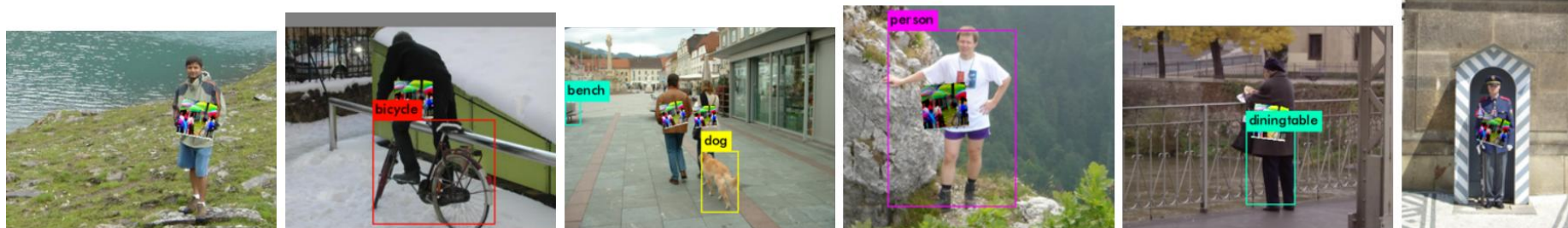
No patch



Random patch



OBJ patch

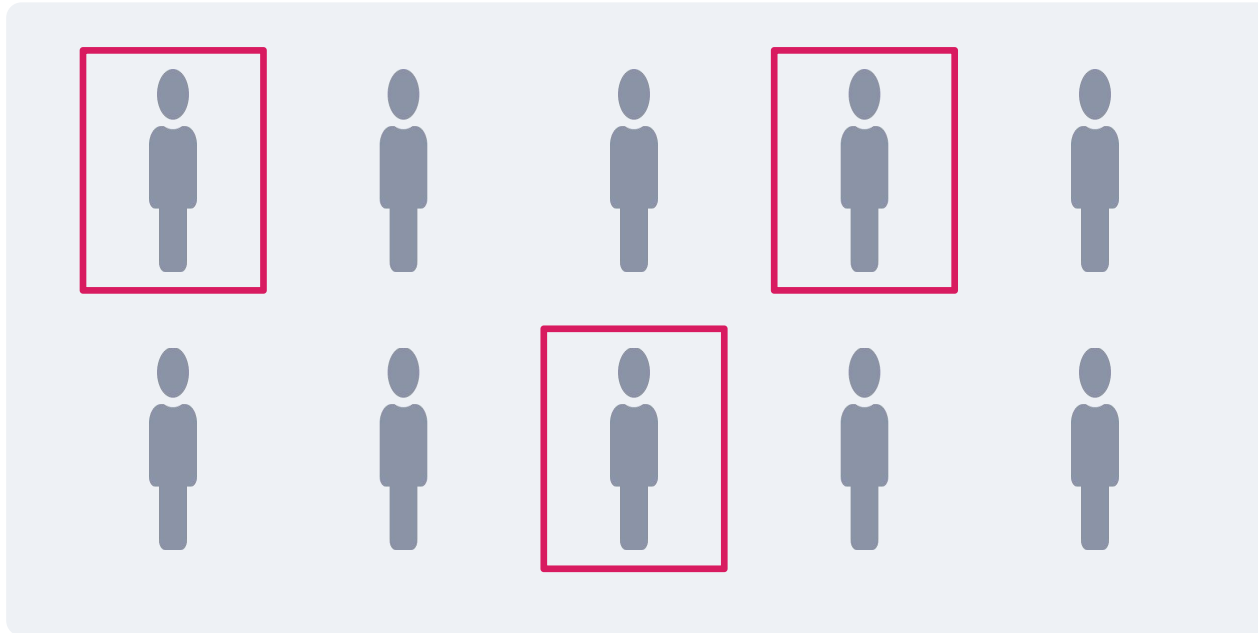


Paper, Fig. 6

Random patch: people are still detected.

OBJ patch: most people vanish. Misses: patch off-center.

Recall: how many people are still found?



10 people, 3 found

$$\text{Recall} = 3 / 10 = 30\%$$

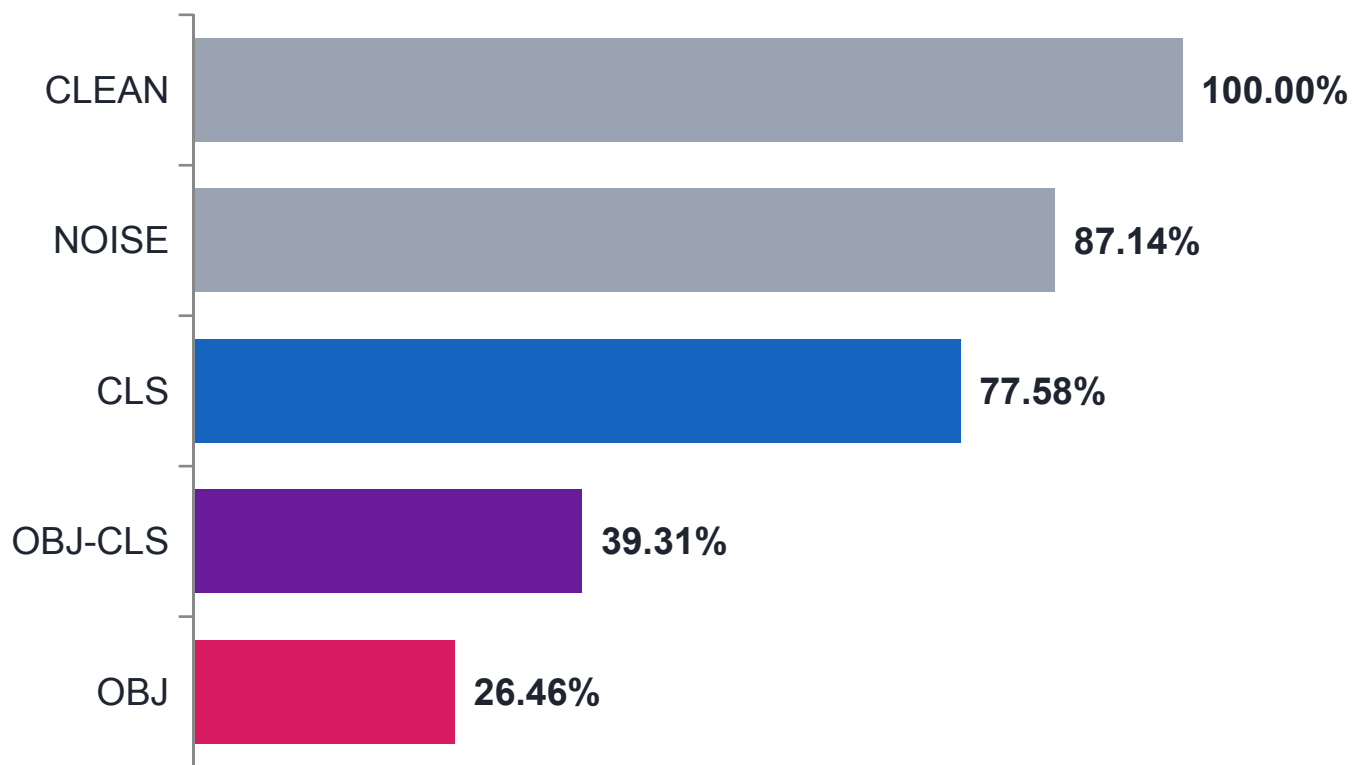
For a camera: how many intruders still set off the alarm?

The attacker wants recall to be LOW. Every missed person is an alarm that never rings.

RESULTS

The OBJ patch removes about 3 out of 4 detections

Recall at threshold 0.4 (paper, Table 1)



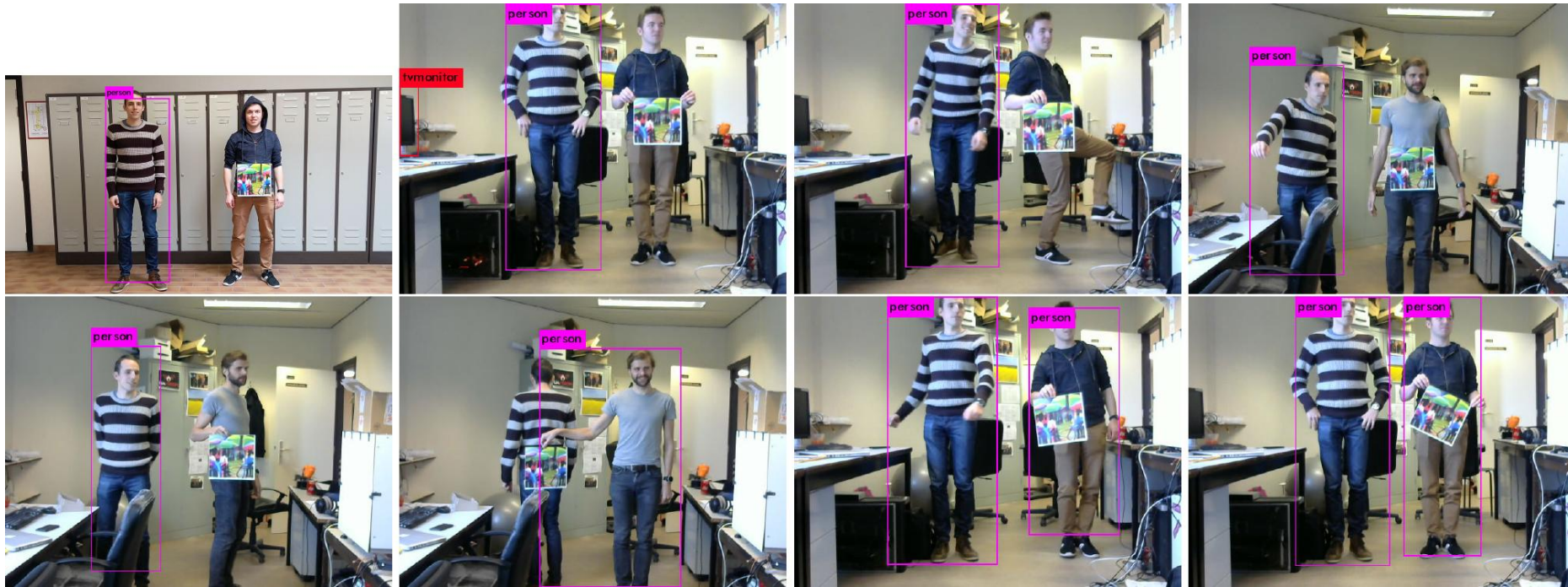
100 → 26

people found with the OBJ patch

Random noise only drops to 87%: it's the pattern, not the covering.

"Nothing here" beats "not a person".

Printed on paper, held in front of a real camera



Paper, Fig. 7

Whoever holds the patch correctly gets no box. When it's tilted (last two frames), he is detected again.

 [Watch the authors' demo video](#)

What the authors proposed, and what happened next

The authors proposed

- Print it on a T-shirt, using a clothing simulation
- Attack several detectors at once: the patch doesn't transfer well to others like Faster R-CNN

Follow-up work

[Adversarial T-shirt](#)

Xu et al., ECCV 2020

A printed T-shirt that still works when the cloth folds and moves.

[Invisibility Cloak](#)

Wu et al., ECCV 2020

Sweaters that hide people from several detectors.

Thank you

Paper: arxiv.org/abs/1904.08653

Code: gitlab.com/EAVISE/adversarial-yolo

